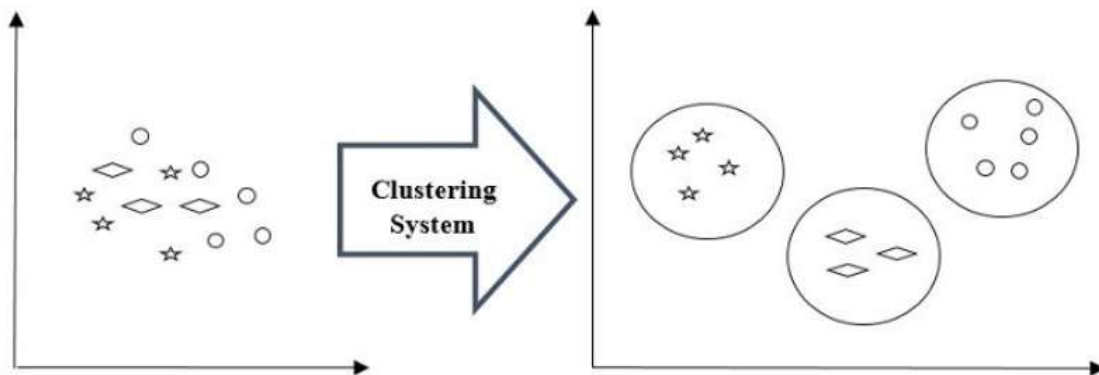


Clustering in Unsupervised ML

Clustering Algorithms are one of the most useful unsupervised machine learning methods. These methods are used to find similarity as well as the relationship patterns among data samples and then cluster those samples into groups having similarity based on features.

For example, below is the diagram which shows clustering system grouped together the similar kind of data in different clusters –



K-Means Clustering Algorithm

K-means clustering algorithm computes the centroids and iterates until it finds optimal centroid. It assumes that the number of clusters are already known. It is also called flat clustering algorithm.

Working of K-Means Algorithm

Step 1. First, we need to specify the number of clusters, K, need to be generated by this algorithm.

Step 2. Next, randomly select K data points and assign each data point to a cluster. In simple words, classify the data based on the number of data points.

Step 3. Now it will compute the cluster centroids.

Step 4. Next, keep iterating the following until we find optimal centroid which is the assignment of data points to the clusters that are not changing any more –

4.1. First, the sum of squared distance between data points and centroids would be computed.

4.2. Now, we have to assign each data point to the cluster that is closer than other cluster (centroid).

4.3. At last compute the centroids for the clusters by taking the average of all data points of that cluster.

Example:

Instance	X	Y
1	185	72
2	170	56
3	168	60
4	179	68
5	182	72
6	188	77

Data Points

Apply K(= 2) - Means algorithm over the data (185, 72), (170, 56), (168, 60), (179,68), (182,72), (188,77) up to two iterations and show the clusters. Initially choose first two objects as initial centroids.

Solution:

Given, number of clusters to be created (K) = 2 say c1 and c2, number of iterations = 2 and first two objects as initial centroids: Centroid for first cluster c1 = (185, 72) and Centroid for second cluster c2 = (170, 56).

Iteration 1: Now calculating similarity by using Euclidean distance measure as:

$$d(c1, 3) = \sqrt{(185 - 168)^2 + (72 - 60)^2} = \sqrt{(17)^2 + (12)^2} = \sqrt{289 + 144} = \sqrt{433}$$

$$d(c2, 3) = \sqrt{(170 - 168)^2 + (56 - 60)^2} = \sqrt{(2)^2 + (-4)^2} = \sqrt{4 + 16} = \sqrt{20}$$

Here, $d(c2, 3) < d(c1, 3)$

So, data point 3 belongs to c2.

$$d(c1, 4) = \sqrt{(185 - 179)^2 + (72 - 68)^2} = \sqrt{(6)^2 + (4)^2} = \sqrt{36 + 16} = \sqrt{52}$$

$$d(c2, 4) = \sqrt{(170 - 179)^2 + (56 - 68)^2} = \sqrt{(-9)^2 + (-12)^2} = \sqrt{81 + 144} = \sqrt{225}$$

Here, $d(c1, 4) < d(c2, 4)$

So, data point 4 belongs to c1.

$$d(c1, 5) = \sqrt{(185 - 182)^2 + (72 - 72)^2} = \sqrt{(3)^2 + (0)^2} = \sqrt{9}$$

$$d(c2, 5) = \sqrt{(170 - 182)^2 + (56 - 72)^2} = \sqrt{(-12)^2 + (-16)^2} = \sqrt{144 + 256} = \sqrt{400}$$

Here, $d(c1, 5) < d(c2, 5)$

So, data point 5 belongs to c1.

$$d(c1, 6) = \sqrt{(185 - 188)^2 + (72 - 77)^2} = \sqrt{(-3)^2 + (-5)^2} = \sqrt{9 + 25} = \sqrt{34}$$

$$d(c2, 6) = \sqrt{(170 - 188)^2 + (56 - 77)^2} = \sqrt{(-18)^2 + (-21)^2} = \sqrt{324 + 441} = \sqrt{765}$$

Here, $d(c1, 6) < d(c2, 6)$

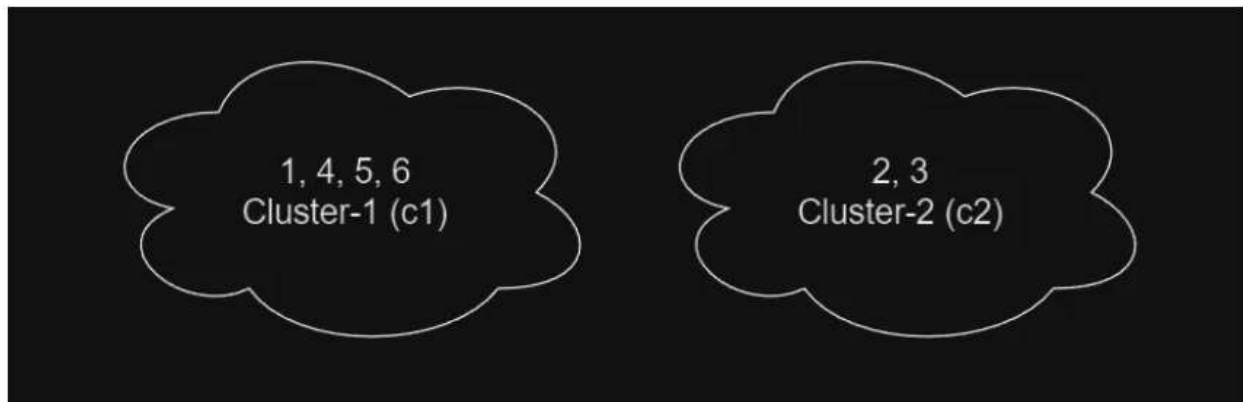
So, data point 6 belongs to c1.

Representing above information in tabular form:

Instance	X	Y	Distance(C1)	Distance(C2)	Cluster
1	185	72			c1
2	170	56			c2
3	168	60	$\sqrt{433}$	$\sqrt{20}$	c2
4	179	68	$\sqrt{52}$	$\sqrt{225}$	c1
5	182	72	$\sqrt{9}$	$\sqrt{400}$	c1
6	188	77	$\sqrt{34}$	$\sqrt{765}$	c1

Distance of each data points from cluster centroids

The resulting cluster after first iteration is:



Data points cluster

Iteration 2: Now calculating centroid for each cluster:

$$\text{Centroid for first cluster } c1 = \left(\frac{185+179+182+188}{4}, \frac{72+68+72+77}{4} \right) = (183.5, 72.25)$$

$$\text{Centroid for second cluster } c2 = \left(\frac{170+168}{2}, \frac{56+60}{2} \right) = (169, 58)$$

Now, again calculating similarity:

$$d(c1, 1) = \sqrt{(183.5 - 185)^2 + (72.25 - 72)^2} = 1.5207$$

$$d(c2, 1) = \sqrt{(169 - 185)^2 + (58 - 72)^2} = 21.2603$$

Here, $d(c1, 1) < d(c2, 1)$

So, data point 1 belongs to $c1$.

$$d(c1, 2) = \sqrt{(183.5 - 170)^2 + (72.25 - 56)^2} = 21.1261$$

$$d(c2, 2) = \sqrt{(169 - 170)^2 + (58 - 56)^2} = 2.2361$$

Here, $d(c2, 2) < d(c1, 2)$

So, data point 2 belongs to $c2$.

$$d(c1, 3) = \sqrt{(183.5 - 168)^2 + (72.25 - 60)^2} = 19.7563$$

$$d(c2, 3) = \sqrt{(169 - 168)^2 + (58 - 60)^2} = 2.2361$$

Here, $d(c2, 3) < d(c1, 3)$

So, data point 3 belongs to $c2$.

$$d(c1, 4) = \sqrt{(183.5 - 179)^2 + (72.25 - 68)^2} = 6.1897$$

$$d(c2, 4) = \sqrt{(169 - 179)^2 + (58 - 68)^2} = 14.1421$$

Here, $d(c1, 4) < d(c2, 4)$

So, data point 4 belongs to $c1$.

$$d(c1, 5) = \sqrt{(183.5 - 182)^2 + (72.25 - 72)^2} = 1.5207$$

$$d(c2, 5) = \sqrt{(169 - 182)^2 + (58 - 72)^2} = 19.1050$$

Here, $d(c1, 5) < d(c2, 5)$

So, data point 5 belongs to $c1$.

$$d(c1, 6) = \sqrt{(183.5 - 188)^2 + (72.25 - 77)^2} = 6.5431$$

$$d(c2, 6) = \sqrt{(169 - 188)^2 + (58 - 77)^2} = 26.8701$$

Here, $d(c1, 6) < d(c2, 6)$

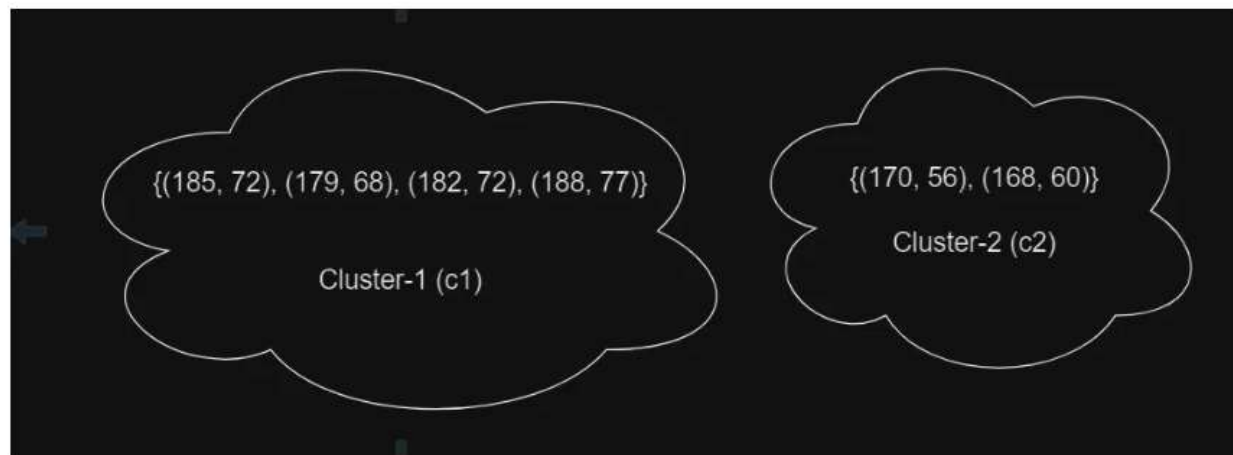
So, data point 6 belongs to c1.

Representing above information in tabular form.

Instance	X	Y	Distance(C1)	Distance(C2)	Cluster
1	185	72	1.5207	21.2603	c1
2	170	56	21.1261	2.2361	c2
3	168	60	19.7563	2.2361	c2
4	179	68	6.1897	14.1421	c1
5	182	72	1.5207	19.105	c1
6	188	77	6.5431	26.8701	c1

Distance of each data points from cluster centroids

The resulting cluster after second iteration is:



Data points cluster

As we have already completed two iteration as asked by our question, the numerical ends here.

Since, the clustering doesn't change after second iteration, so terminate the iteration even if question doesn't say so.

DBSCAN Clustering in ML - Density based clustering

DBSCAN is a density-based clustering algorithm that groups data points that are closely packed together and marks outliers as noise based on their density in the feature space. It identifies clusters as dense regions in the data space separated by areas of lower density. Unlike K-Means or hierarchical clustering which assumes clusters are compact and spherical, DBSCAN perform well in handling real-world data irregularities such as:

Arbitrary-Shaped Clusters: Clusters can take any shape not just circular or convex.

Noise and Outliers: It effectively identifies and handles noise points without assigning them to any cluster.

How Does DBSCAN Work?

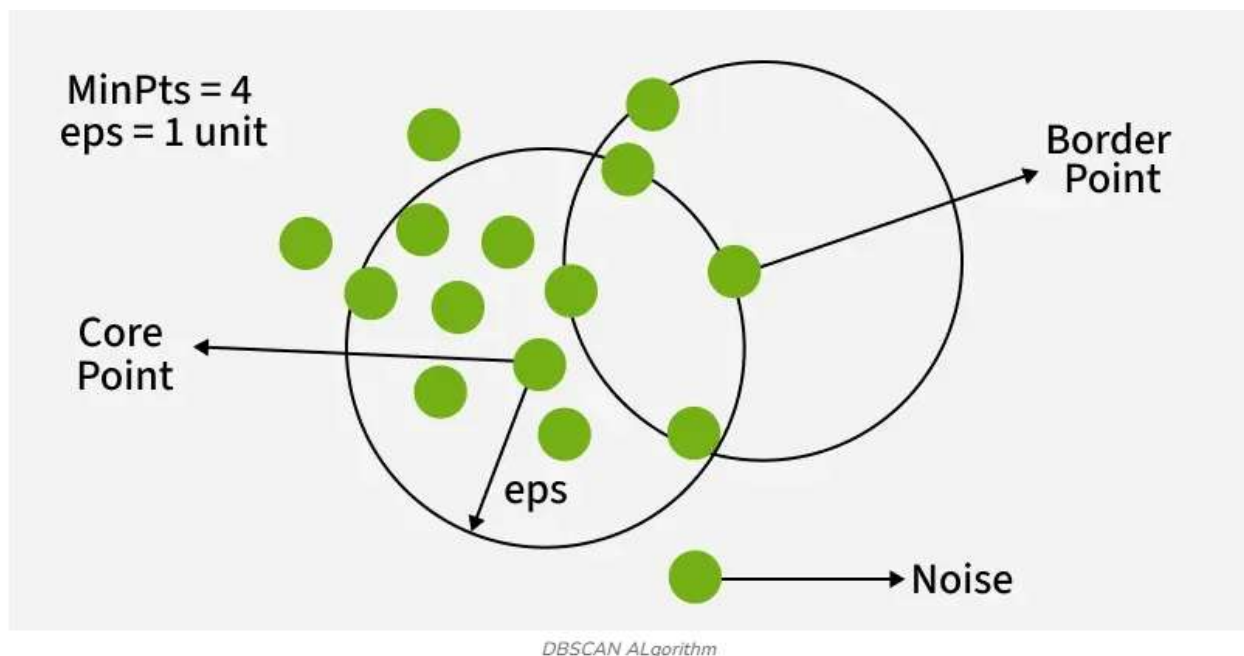
DBSCAN works by categorizing data points into three types:

Core points: which have a sufficient number of neighbors within a specified radius (epsilon)

Border points: which are near core points but lack enough neighbors to be core points themselves

Noise points: which do not belong to any cluster.

By iteratively expanding clusters from core points and connecting density-reachable points, DBSCAN forms clusters without relying on rigid assumptions about their shape or size.



Example

Given the points A(3, 7), B(4, 6), C(5, 5), D(6, 4), E(7, 3), F(6, 2), G(7, 2) and H(8, 4), Find the core points and outliers using DBSCAN. Take $\text{Eps} = 2.5$ and $\text{MinPts} = 3$.

Solution:

Given, Epsilon(Eps) = 2.5

Minimum Points(MinPts) = 3

Let's represent the given data points in tabular form:

Data Points	X	Y
A	3	7
B	4	6
C	5	5
D	6	4
E	7	3
F	6	2
G	7	2
H	8	4

Data points

Step 1: To find the core points, outliers and clusters by using DBSCAN we need to first calculate the distance among all pairs of given data point. Let us use Euclidean distance measure for distance calculation.

The final distance matrix becomes as shown below:

Data Points	A	B	C	D	E	F	G	H
A	0	1.4142	2.8284	4.2426	5.6569	5.831	6.4031	5.831
B		0	1.4142	2.8284	4.2426	4.4721	5	4.4721
C			0	1.4142	2.8284	3.1623	3.6056	3.1623
D				0	1.4142	2	2.2361	2
E					0	1.4142	1	1.4142
F						0	1	2.8284
G							0	2.2361
H								0

Proximity matrix

The diagonal elements of this matrix will always be 0 as the distance of a point with itself is always 0. In the above table, Distance $\leq$ Epsilon (i.e. 2.5) is marked red.

Step 2: Now, finding all the data points that lie in the Eps-neighborhood of each data points. That is, put all the points in the neighborhood set of each data point whose distance is ≤ 2.5 .

$N(A) = \{B\}$; — — — — — $\rightarrow$ because distance of B is ≤ 2.5 with A

$N(B) = \{A, C\}$; — — — — — $\rightarrow$ because distance of A and C is ≤ 2.5 with B

$N(C) = \{B, D\}$; — — — — — $\rightarrow$ because distance of B and D is ≤ 2.5 with C

$N(D) = \{C, E, F, G, H\}$; — $\rightarrow$ because distance of C, E, F, G and H is ≤ 2.5 with D

$N(E) = \{D, F, G, H\}$; — — $\rightarrow$ because distance of D, F, G and H is ≤ 2.5 with E

$N(F) = \{D, E, G\}$; — — — — $\rightarrow$ because distance of D, E and G is ≤ 2.5 with F

$N(G) = \{D, E, F, H\}$; — — $\rightarrow$ because distance of D, E, F and H is ≤ 2.5 with G

$N(H) = \{D, E, G\}$; — — — — $\rightarrow$ because distance of D, E and G is ≤ 2.5 with H

Here, data points A, B and C have neighbors $\leq \text{MinPts}$ (i.e. 3) so can't be considered as core points. Since they belong to the neighborhood of other data points, hence there exist no outliers in the given set of data points.

Association

Association is a type of unsupervised learning that finds patterns or relationships between items in a dataset.

Imagine you have data from a supermarket. If many customers buy bread and butter together, association analysis might find a rule like: "If a customer buys bread, they are likely to buy butter too." This is called an association rule.

Common algorithms: Apriori, FP-Growth

Used in: Market basket analysis, Recommendation systems, Web usage mining